

✓ Suicide Detection ML Pipeline

This notebook loads Suicide_Detection.csv and performs EDA, feature engineering, TF-IDF modeling, and performance comparison.

Double-click (or enter) to edit

✓ 1. Import Libraries

```
1 import pandas as pd
2 import numpy as np
3 import re, string
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
8 from sklearn.model_selection import train_test_split
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.ensemble import RandomForestClassifier
11 from sklearn.neighbors import KNeighborsClassifier
12 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusi
```

✓ 2. Load Dataset

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3 df = pd.read_csv('/content/drive/MyDrive/Suicide_Detection.csv', index_col=0)
4 # df = pd.read_csv("Suicide_Detection.csv")
5 df.columns = ["text", "label"]
6 df.head()
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

	text	label
2	Ex Wife Threatening SuicideRecently I left my ...	suicide
3	Am I weird I don't get affected by compliments...	non-suicide
4	Finally 2020 is almost over... So I can never ...	non-suicide
8	i need helpjust help me im crying so hard	suicide
9	I'm so lostHello, my name is Adam (16) and I've...	suicide

✓ 3. Clean Text

```
1 def normalize(text):
2     text = text.lower()
3     text = re.sub(r"http\S+|www\S+", "", text)
4     text = re.sub(r"\s+", " ", text)
5     text = text.translate(str.maketrans("", "", string.punctuation))
6     return text.strip()
7
8 df["clean_text"] = df["text"].apply(normalize)
9 df.head()
```

	text	label	clean_text
2	Ex Wife Threatening SuicideRecently I left my ...	suicide	ex wife threatening suiciderrecently i left my ...
3	Am I weird I don't get affected by compliments...	non-suicide	am i weird i dont get affected by compliments ...
4	Finally 2020 is almost over... So I can never ...	non-suicide	finally 2020 is almost over so i can never hea...
8	i need helpjust help me im crying so hard	suicide	i need helpjust help me im crying so hard
9	I'm so lostHello, my name is Adam (16) and I've...	suicide	i'm so losthello my name is adam 16 and i've b...

4. EDA: Length Features

```

1 df["char_length"] = df["clean_text"].str.len()
2 df["word_length"] = df["clean_text"].str.split().apply(len)
3
4 df.groupby("label")["word_length"].describe()

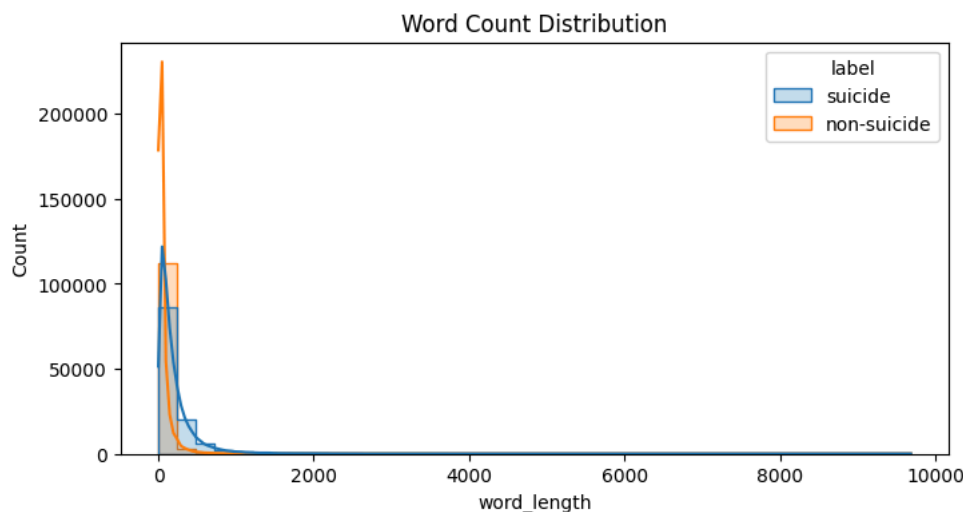
```

	count	mean	std	min	25%	50%	75%	max
label								
non-suicide	116037.0	60.674802	134.110831	0.0	18.0	31.0	60.0	8220.0
suicide	116037.0	202.343796	254.904606	0.0	60.0	127.0	250.0	9684.0

```

1 plt.figure(figsize=(8,4))
2 sns.histplot(df, x="word_length", hue="label", bins=40, element="step", kde=True)
3 plt.title("Word Count Distribution")
4 plt.show()

```



5. Token Frequency Analysis

```

1 cv = CountVectorizer(stop_words="english")
2 X_counts = cv.fit_transform(df["clean_text"])
3 tokens = np.array(cv.get_feature_names_out())
4
5 def top_tokens(label, n=20):
6     mask = df["label"] == label
7     freq = X_counts[mask.values].sum(axis=0).A1
8     idx = freq.argsort()[::-1][:n]
9     return pd.DataFrame({"token": tokens[idx], "count": freq[idx]})
10
11 top_tokens("suicide", 15)

```

	token	count
0	im	213466
1	just	203484
2	dont	139932
3	like	131021
4	want	128286
5	life	110848
6	feel	107942
7	know	106346
8	ive	75380
9	people	71895
10	time	69737
11	really	65041
12	think	57359
13	going	57190
14	friends	48499

6. Sentiment-like Lexical Features

```

1 neg_words = {"suicide", "kill", "die", "cut", "hurt", "hopeless", "worthless", "end", "empty"}
2 pos_words = {"love", "happy", "great", "hope", "enjoy", "friends", "support", "smile", "proud"}
3
4 def lex_features(text):
5     ws = text.split()
6     pos = sum(w in pos_words for w in ws)
7     neg = sum(w in neg_words for w in ws)
8     return pd.Series([pos, neg, pos-neg])
9
10 df[["pos_score", "neg_score", "sent_balance"]] = df["clean_text"].apply(lex_features)
11 df.groupby("label")[["pos_score", "neg_score", "sent_balance"]].mean()

```

	pos_score	neg_score	sent_balance
label			
non-suicide	0.310909	0.103183	0.207727
suicide	1.176323	1.495954	-0.319631

7. Train/Test Split

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     df["clean_text"], df["label"],
3     test_size=0.2, stratify=df["label"], random_state=42
4 )

```

8. Glove embeddings

```

!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

```

```

1 import os
2
3 if not os.path.exists("glove.6B.zip"):
4     !wget http://nlp.stanford.edu/data/glove.6B.zip

```

```

5     !unzip glove.6B.zip
6     print("GloVe embeddings downloaded and unzipped.")
7 else:
8     print("GloVe embeddings already present.")

```

```

--2025-12-09 16:48:54-- http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2025-12-09 16:48:55-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2025-12-09 16:48:55-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip'

glove.6B.zip      100%[=====>] 822.24M  3.89MB/s   in 3m 7s

2025-12-09 16:52:03 (4.39 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

Archive: glove.6B.zip
  inflating: glove.6B.50d.txt
  inflating: glove.6B.100d.txt
  inflating: glove.6B.200d.txt
  inflating: glove.6B.300d.txt
GloVe embeddings downloaded and unzipped.

```

```

1 embedding_dim = 100
2
3 word_to_vec_map = {}
4 with open(f'glove.6B.{embedding_dim}d.txt', encoding='utf-8') as f:
5     for line in f:
6         values = line.split()
7         word = values[0]
8         vec = np.asarray(values[1:], dtype='float32')
9         word_to_vec_map[word] = vec
10
11 print(f"Loaded {len(word_to_vec_map)} word vectors.")
12
13 def get_sentence_embedding(text, word_to_vec_map, embedding_dim):
14     words = str(text).lower().split() # Token+ lowercase
15     embedding = np.zeros(embedding_dim)
16     num_words = 0
17     for word in words:
18         if word in word_to_vec_map:
19             embedding += word_to_vec_map[word]
20             num_words += 1
21     if num_words > 0:
22         return embedding / num_words
23     else:
24         return embedding #zero for null sentences or unknown words
25
26
27 X_train, X_test, y_train, y_test = train_test_split(
28     df["clean_text"], df["label"],
29     test_size=0.2, stratify=df["label"], random_state=42
30 )
31
32 X_train_glove = np.array([get_sentence_embedding(text, word_to_vec_map, embedding_dim) for text in X_train])
33 X_test_glove = np.array([get_sentence_embedding(text, word_to_vec_map, embedding_dim) for text in X_test])
34
35 print(f"X_train_glove shape: {X_train_glove.shape}")
36 print(f"X_test_glove shape: {X_test_glove.shape}")

```

```

Loaded 400000 word vectors.
X_train_glove shape: (185659, 100)
X_test_glove shape: (46415, 100)

```

9. Models: Logistic Regression and Random Forest

```

1 models = {
2     "Logistic Regression": LogisticRegression(max_iter=300),
3     "Random Forest": RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1),
4     "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5)
5 }
6
7 results = {}
8
9 for name, model in models.items():
10     model.fit(X_train_glove, y_train)
11     preds = model.predict(X_test_glove)
12     probs = model.predict_proba(X_test_glove)[: ,1]
13
14     results[name] = {
15         "accuracy": accuracy_score(y_test, preds),
16         "precision": precision_score(y_test, preds, pos_label="suicide"),
17         "recall": recall_score(y_test, preds, pos_label="suicide"),
18         "f1": f1_score(y_test, preds, pos_label="suicide"),
19         "roc_auc": roc_auc_score((y_test=='suicide').astype(int), probs),
20         "conf_matrix": confusion_matrix(y_test, preds)
21     }
22
23 results

```

```

{'Logistic Regression': {'accuracy': 0.8601745125498222,
  'precision': 0.8346746746746747,
  'recall': 0.8982634549920283,
  'f1': 0.8653023950853016,
  'roc_auc': np.float64(0.9202725468535083),
  'conf_matrix': array([[19079,  4129],
    [ 2361, 20846]])},
 'Random Forest': {'accuracy': 0.8712700635570397,
  'precision': 0.8977839335180056,
  'recall': 0.837936829404921,
  'f1': 0.8668286268304098,
  'roc_auc': np.float64(0.9437730958519436),
  'conf_matrix': array([[20994,  2214],
    [ 3761, 19446]])},
 'K-Nearest Neighbors': {'accuracy': 0.7909081116018528,
  'precision': 0.7158800204655922,
  'recall': 0.9646658335846943,
  'f1': 0.8218579636190092,
  'roc_auc': np.float64(0.8901853655291605),
  'conf_matrix': array([[14323,  8885],
    [ 820, 22387]])}}

```

10. Feature Importance (Logistic Regression)

```

1 lr = models["Logistic Regression"]
2
3 coef = lr.coef_[0]
4 print("Logistic Regression Coefficients for GloVe dimensions (first 10):", coef[:10])
5 print("Logistic Regression Coefficients for GloVe dimensions (last 10):", coef[-10:])

```

```

Logistic Regression Coefficients for GloVe dimensions (first 10): [-0.02431171 -2.88887259 -3.50103048 -0.53836269  0.95453832
-1.45798437 -1.64162883  3.38479192 -0.75813411]
Logistic Regression Coefficients for GloVe dimensions (last 10): [-2.06169332 -2.75824625  5.17020646  2.49852732 -2.54461992  1
 2.61736669  0.15549258  0.9769602  1.6043219 ]

```

11. BERT

```

1 from google.colab import userdata
2 userdata.get('HF_demo')

```

```

'hf_VQAFcnoASEavDLnHDoGzYVwroY1RHzfYhn'

```

```

1 # 1) Install packages (run once)
2 !pip install -q transformers datasets huggingface_hub accelerate
3
4 # 2) Authentication - No need now, using dataset from drive
5 # import os
6 # from huggingface_hub import login
7 # hf_token = os.environ.get("HF_TOKEN") or os.environ.get("HUGGINGFACEHUB_API_TOKEN") or None
8 # if hf_token:
9 #     login(token=hf_token)
10 # else:
11 #     login()
12
13 # 3) PyTorch model class. [MUST CUDA]
14 import torch
15 from datasets import Dataset # Import Dataset
16 from transformers import (
17     AutoTokenizer,
18     AutoModelForSequenceClassification,
19     TrainingArguments,
20     Trainer,
21     DataCollatorWithPadding,
22     EvalPrediction,
23 )
24
25 device = "cuda" if torch.cuda.is_available() else "cpu"
26 print("Using device:", device)
27
28 # 4) Load suicide detection dataset
29 # Convert pandas Series to Hugging Face Dataset format
30 train_df = pd.DataFrame({'text': X_train, 'label': y_train})
31 test_df = pd.DataFrame({'text': X_test, 'label': y_test})
32
33 raw_train_dataset = Dataset.from_pandas(train_df)
34 raw_test_dataset = Dataset.from_pandas(test_df)
35
36 # Map string labels to integers
37 label_mapping = {'non-suicide': 0, 'suicide': 1}
38 num_labels = len(label_mapping) # 2 for binary classification
39
40 def map_labels_to_int(examples):
41     # Ensure label mapped
42     examples['label'] = label_mapping[examples['label']]
43     return examples
44
45 # Corrected column name to remove: '__index_level_0__'
46 train_dataset_mapped = raw_train_dataset.map(map_labels_to_int, remove_columns=["__index_level_0__"])
47 test_dataset_mapped = raw_test_dataset.map(map_labels_to_int, remove_columns=["__index_level_0__"])
48
49 # 5) Tokenizer + model
50 model_name = "bert-base-uncased"
51 tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
52
53 def preprocess_function(examples):
54     return tokenizer(examples["text"], truncation=True, max_length=256)
55
56 # Tokenize the dataset and remove the original 'text' column
57 tokenized_train_dataset = train_dataset_mapped.map(preprocess_function, batched=True, remove_columns=['text'])
58 tokenized_test_dataset = test_dataset_mapped.map(preprocess_function, batched=True, remove_columns=['text'])
59
60 data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
61
62 # 6) Load model (PyTorch)
63 # The warning "Some layers ..." is expected
64 model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=num_labels)
65 model.to(device)
66
67 # 7) Trainer setup
68 from transformers import TrainingArguments
69
70 training_args = TrainingArguments(

```

```

71 output_dir="hf_bert_run",
72 num_train_epochs=1,
73
74
75 per_device_train_batch_size=32, # No OOM on T4 gpu
76 per_device_eval_batch_size=64, # reduce eval wall-time
77 gradient_accumulation_steps=1, # No OOM
78
79 #Mixed precision -> speed
80 fp16=True,
81
82 #fastest single epoch
83 eval_strategy="no", # skip evaluation
84 save_strategy="no", # skip checkpoints
85 logging_strategy="no", # skip periodic logging
86
87 #regularization
88 learning_rate=2e-5,
89 weight_decay=0.01,
90
91 # runtime helpers
92 dataloader_num_workers=4, # 4 processes for loading
93 dataloader_pin_memory=True, # GPU transfer
94 remove_unused_columns=True,
95 load_best_model_at_end=False,
96 push_to_hub=False,
97 )
98
99
100 def compute_metrics(p: EvalPrediction):
101     preds = p.predictions.argmax(-1)
102     accuracy = (preds == p.label_ids).astype(float).mean().item()
103     return {"accuracy": accuracy}
104
105 trainer = Trainer(
106     model=model,
107     args=training_args,
108     train_dataset=tokenized_train_dataset, #actual train dataset
109     eval_dataset=tokenized_test_dataset, #actual test dataset
110     tokenizer=tokenizer, # tokenizer for data_collator
111     data_collator=data_collator,
112     compute_metrics=compute_metrics,
113 )
114
115 # 8) Train
116 trainer.train()
117
118 # 9) Save model
119 trainer.save_model("fine_tuned_bert")
120 tokenizer.save_pretrained("fine_tuned_bert")
121 print("Saved to /content/fine_tuned_bert")

```



Copy a token from [your Hugging Face tokens page](#) and paste it below.

Immediately click login after copying your token or it might be stored in plain text in

this notebook file.

Token:

☒ Add token as git credential?

Login

```

1 predictions = trainer.predict(tokenized_test_dataset)
2
3 # Get predicted labels (0 or 1)
4 preds_bert_int = predictions.predictions.argmax(-1)
5
6 # Get true labels (0 or 1)
7 y_test_bert_int = predictions.label_ids
8
9 # Get probabilities for the positive class (suicide, which is label 1)
10 # The predictions are logits, apply softmax to get probabilities
11 # Using torch.softmax for numerical stability and correctness
12 probs_bert = torch.softmax(torch.tensor(predictions.predictions), dim=1)[ :, 1].numpy()
13
14 # Map integer labels back to string labels for consistency with previous metrics
15 # Reverse the label_mapping: {0: 'non-suicide', 1: 'suicide'}
16 reverse_label_mapping = {v: k for k, v in label_mapping.items()}
17 y_test_bert_str = np.array([reverse_label_mapping[label] for label in y_test_bert_int])
18 preds_bert_str = np.array([reverse_label_mapping[label] for label in preds_bert_int])
19
20 # Calculate metrics
21 results["BERT"] = {
22     "accuracy": accuracy_score(y_test_bert_str, preds_bert_str),
23     "precision": precision_score(y_test_bert_str, preds_bert_str, pos_label="suicide"),
24     "recall": recall_score(y_test_bert_str, preds_bert_str, pos_label="suicide"),
25     "f1": f1_score(y_test_bert_str, preds_bert_str, pos_label="suicide"),
26     "roc_auc": roc_auc_score((y_test_bert_str == 'suicide').astype(int), probs_bert),
27     "conf_matrix": confusion_matrix(y_test_bert_str, preds_bert_str)
28 }
29
30 print("BERT Model Performance:")
31 for metric, value in results["BERT"].items():
32     print(f" {metric.replace('_', ' ').capitalize()}: {value}")
33

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker pr
warnings.warn(

BERT Model Performance:

```

Accuracy: 0.9848971237746418
Precision: 0.987818623200971
Recall: 0.9819020123238678
F1: 0.9848514316585629
Roc auc: 0.9987816569775546
Conf matrix: [[22927  281]
 [ 420 22787]]

```